

Congestion-aware clustering

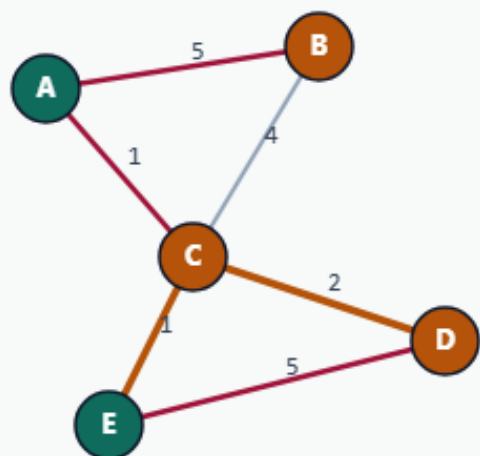
When bridge edges are congested, cutting them becomes expensive

Start from BAD_SEED

CONGESTION-AWARE CLUSTERING

Start from BAD_SEED

Step 1 / 5 · bad-seed · module04-01-congestion-aware-clustering



Explanation

Same cutsize-12 seed $AE|BCD$. Congestion map marks $C-D=5$ and $C-E=4$ — cheap wire cuts may be expensive for routing.

- plain cut ignores congestion
- penalty sums cong on cut edges
- λ trades plain vs penalty

seed cut: 12
cong: $C|D=5$, $C|E=4$

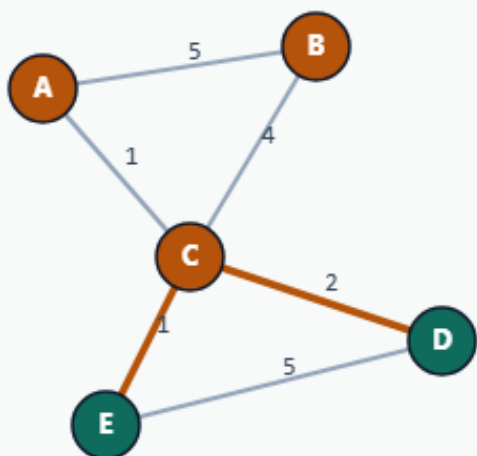
Start from BAD_SEED

$\lambda=0 \rightarrow$ ordinary FM

CONGESTION-AWARE CLUSTERING

$\lambda=0 \rightarrow$ ordinary FM

Step 2 / 5 · lam0 · module04-01-congestion-aware-clustering



Explanation

With $\lambda=0$, boosted weights equal original weights. FM recovers ABC|DE: plain=3 but penalty=9 because both congested bridges are cut.

- plain=3, pen=9
- parts: ABC|DE
- Objective ignores routing pain

$\lambda=0$
plain=3
pen=9

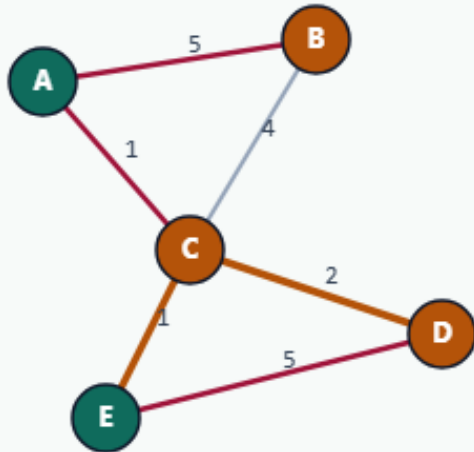
$\lambda=0 \rightarrow$ ordinary FM

Boost weights by $\lambda \cdot \text{cong}$

CONGESTION-AWARE CLUSTERING

Boost weights by $\lambda \cdot \text{cong}$

Step 3 / 5 · boost · module04-01-congestion-aware-clustering



Explanation

For $\lambda=5$, C-D and C-E become very expensive to cut. FM optimizes the boosted graph, then we report plain cut and congestion penalty separately.

- $w' = w + \lambda \cdot \text{cong}$
- Run FM on w'
- Score plain + $\lambda \cdot \text{pen}$ on original

$\lambda=5$

C-D and C-E heavily boosted

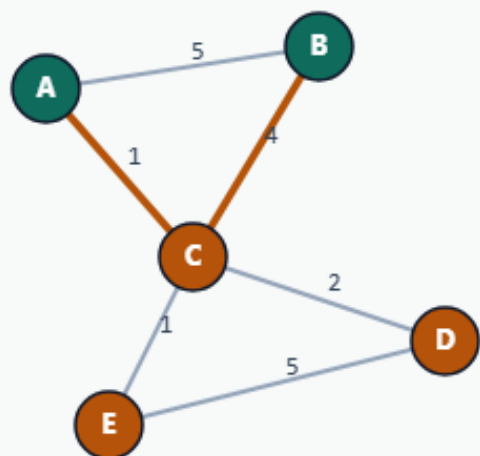
Boost weights by $\lambda \cdot \text{cong}$

$\lambda=5 \rightarrow \text{pen } 0, \text{ plain } 5$

CONGESTION-AWARE CLUSTERING

$\lambda=5 \rightarrow \text{pen } 0, \text{ plain } 5$

Step 4 / 5 · lam5 · module04-01-congestion-aware-clustering



Explanation

Result AB|CDE: plain cut rises to 5, but congestion penalty drops to 0 — congested bridges stay internal.

- parts: AB|CDE
- plain=5, pen=0
- objective=5

$\lambda=5$
plain=5
pen=0

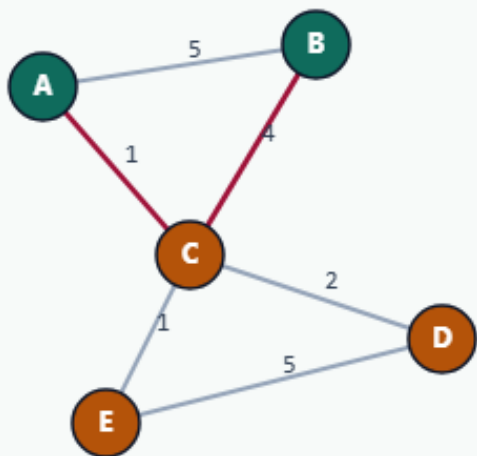
$\lambda=5 \rightarrow \text{pen } 0, \text{ plain } 5$

Objective tradeoffs

CONGESTION-AWARE CLUSTERING

Objective tradeoffs

Step 5 / 5 · takeaway · module04-01-congestion-aware-clustering



Explanation

EDA flows rarely optimize cut alone. λ makes the congestion tax explicit so students see the Pareto move between wire and routing.

- $\lambda=0$ favors classic communities
- $\lambda>0$ protects congested edges
- Same FM engine, different weights

Goldens: (3,9) vs (5,0)

Browser lab track

- In the browser lab, compare lambda zero with lambda five from the same bad seed
- Watch the penalty drop to zero when the congested bridge is avoided

Implement track

- Run congestion-aware partition at lambda zero and five
- Confirm plain cut three with penalty nine at lambda zero

Implement track — try these

```
export PYTHONPATH=../common
```

```
python ../common/solvers.py examples/tiny_graph.json --mode congestion  
--seed ../module02-05-kernighan-lin/examples/seed_partition.json  
--congestion examples/congestion.json --lambda 5
```

Pitfalls to watch

- Applying congestion only after FM misses the point, weights must change before moves
- Huge λ can ignore connectivity entirely

Your turn

- Match both lambda goldens